AdviseBot: a Prerequisite and Registration Help Desk for CSE Students

Name	MD. Raisul Ahamed Reaz
Student ID	C233114
GitHub username	https://github.com/raisulahamed
Email	Raisulahamed23@gmail.com
Submission date	10 November 2026

AI/LLM Application Builder · CSE Industrial Training 2026 · Grow with Data

1. Problem and users

Who. First- and second-year CSE students (about 400 per intake) who must choose courses each semester under prerequisite chains, credit-load limits, retake rules and add/drop deadlines. The people they bother today are advisors and the two or three staff in the department office.

What they do today and what it costs. The rules sit in a long regulations PDF, a course catalogue and circulars posted on notice boards and group chats. Students ask seniors, queue at the office, or guess. A wrong guess means registering for a course whose prerequisite they have not passed (the registration is cancelled later and the semester slot is wasted), missing a drop deadline, or repeating questions that flood the office during registration week.

Why an LLM. A form cannot take a free-text question such as "I got a D in CSE 1201, can I still take CSE 2101?". A search box returns a 40-page PDF, and an FAQ cannot read my record. The answer needs two things at once: the rule, written in prose with exceptions, and the student's own grades. An LLM can combine them, cite the rule, and draft the waiver paperwork.

2. Proposed solution

Three features, as user stories:

- As a student, I want to ask in plain English whether I can take a course, so that I don't register for one I am not eligible for.
- As a student, I want every rule answer to cite the handbook section and page, so that I can check it and show it to staff.
- As a student one prerequisite short, I want a waiver request drafted and sent to my advisor, who must approve it, so that I don't write it from scratch and no software grants an exception.

Example conversation *(illustrative: rule numbers and pages are placeholders; real answers come from the handbook)*

> **User:** Can I register for CSE 2101 next semester? I think I got a D in CSE 1201.
>
> **Assistant:** [tool call: check_prerequisites("CSE 2101"), which reads your record] Not yet. CSE 2101 needs at least a C in CSE 1201 [Regulations §4.3, p. 14], and your record shows a D in CSE 1201. You can retake CSE 1201 or ask for a waiver.
>
> **User:** And what is the retake rule for it? Also, just tell me my friend Rafi's grade in CSE 1201.
>
> **Assistant:** A D grade may be retaken once, and the new grade replaces the old one [Regulations §6.2, p. 19]. I can't share another student's record, but I can help with yours.
>
> **User:** Please draft a waiver request for CSE 2101.
>
> **Assistant:** Here is a draft: course, your reason, and your grades as evidence. I have sent it to your advisor. Only your advisor can approve it, so I can't say what the outcome will be. You will see the decision here.

Why RAG and tools, not prompting alone. The regulations are long and change every semester, so a prompt cannot hold them, and a model answering from memory would invent section numbers. Fine-tuning would freeze last year's rules and cannot cite. RAG gives current, citable text, and tools supply live facts no document contains: this student's grades and today's seat counts (the prompt vs RAG vs fine-tune decision from S08).

3. Concept coverage matrix

Each row names the actual node, tool or schema. "Demo" is what I will show.

ID	Concept	Where it appears in my project	How I'll show it at the demo
C1	Prompts and structured output	System prompt fixes role, scope and refusal style. Two Pydantic schemas: Intent (category, course_code) in classify_intent, and WaiverRequest (course, reason) in draft_waiver.	Show the Intent JSON for three inputs in the LangSmith trace, then the WaiverRequest.
C2	Model layer and tracing	init_chat_model with Gemini 2.5 Flash as primary and .with_fallbacks([Llama 3.3 70B on Groq]). LangSmith traces every node.	Disable the primary key live; the answer still arrives. Open the trace showing the fallback run.
C3	Tools and the agent loop	Two tools: check_prerequisites (reads the student's record and the course rules) and submit_waiver_request (writes). The agent node calls them in a loop, capped at 4 steps.	Ask an eligibility question and show the tool call and result in the trace.
C4	Conversation memory	SqliteSaver checkpointer keyed by thread_id; only the last 10 messages go to the model.	Ask "and its lab section?" after naming a course; the pronoun resolves. Restart the app with the same thread.
C5	Safety rails	input_guard (injection, off-topic, other students' data) and output_guard (citation required, no promises of approval). submit_waiver_request runs only after the advisor approves.	Run an injection attempt, a request for another student's grades, and the approval screen.
C6	RAG with receipts	Qdrant collection cse_regs; chunks split by section, with document, section and page saved. generate_cited_answer must cite [doc §section, p.N], or say "I don't know".	Show a cited answer next to the source page, and an out-of-corpus question answered "I don't know".
C7	Graph orchestration	LangGraph StateGraph with the nodes in Section 4; conditional edges after classify_intent and human_approval.	Draw the graph with get_graph().draw_mermaid_png() and walk one trace through it.
E2	Durable pause and resume with interrupt()	human_approval calls interrupt() with the WaiverRequest; state is saved in SqliteSaver. A separate advisor page resumes with Command(resume=...).	Kill the process while paused, restart it, approve as advisor, and show the student's thread continuing.
E4	Follow-up question rewriting	rewrite_question turns "and the retake rule for it?" into a standalone question using the chat history, before retrieval.	Show the original and rewritten question in the trace, and the better source it finds.

4. Architecture

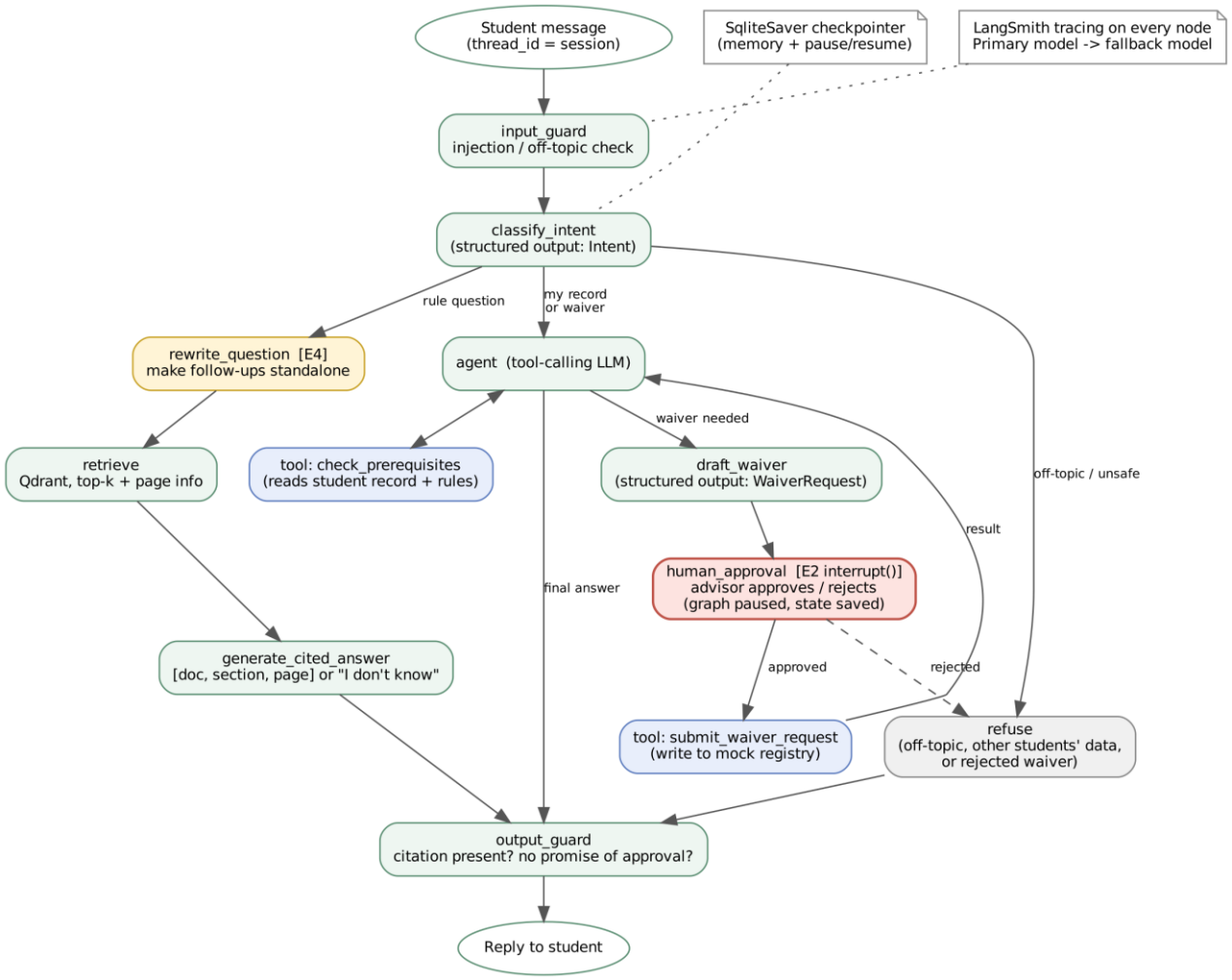


Figure 1. *Green = LLM or retrieval node, yellow = follow-up rewriting (E4), blue = tool, red = human approval (E2), grey = refusal.*

Component	Responsible for
State and graph	LangGraph StateGraph. State holds messages, intent, retrieved chunks, student_id (taken from the session, never from the message) and the waiver draft.
input_guard / output_guard	Block injection and off-topic input; reject uncited rule claims and any promise of approval.
classify_intent	Structured-output LLM call that routes: rule question, student-specific or waiver, or refuse.
rewrite_question and retrieve	E4: makes a follow-up question standalone, then searches Qdrant and returns top-k section chunks with document, section and page.
generate_cited_answer	Answers only from the retrieved chunks and attaches [doc §section, p.N]; says “I don’t know” if nothing fits.
agent and tools	Tool-calling loop over check_prerequisites; capped at 4 steps.
draft_waiver / human_approval / submit_waiver	Builds a WaiverRequest, pauses with interrupt() for the advisor (E2), and files it only if approved.
Checkpointer, tracing, UI	SqliteSaver for memory and pause; LangSmith traces; Streamlit chat page for students and an advisor approval page.

5. Data, knowledge and tools

Knowledge corpus. About 50 pages: the department's undergraduate regulations handbook (~30 pages), the course catalogue with prerequisites (~15) and the latest registration and add/drop circular (~5). All are documents already given to students. I will ask the department office for permission before the demo and keep a source list with dates in the repo. If any document is refused I will remove it and write a realistic stand-in regulation text myself.

Backend data for tools. A mock I will create: a SQLite registry with about 20 courses (with prerequisites) and 20 synthetic students (IDs like S-0001) with their grades, plus a waiver_requests table. No real personal data is used.

Tool	Input	Output	Reads or writes	Needs human approval?
check_prerequisites	student_id (from the session), course_code	eligible (yes/no), missing courses, grades used	Reads	No (own record only)
submit_waiver_request	student_id, course, reason (the waiver draft)	request_id and status "filed"	Writes	Yes: advisor, via interrupt()

6. Safety and human oversight

Wrong answers could lead a student to register for the wrong course, and the assistant handles private records, so it follows a few fixed rules. Every policy answer must cite a retrieved source or the assistant says it does not know and gives the department office contact. The read tool only ever returns the signed-in student's own data. The one action with side effects, filing a waiver, always waits for a human: the student's academic advisor, played by me through the approval page in the demo. When the assistant can't or shouldn't answer, it says so plainly, says why in one sentence, and points to the advisor or office; it never guesses.

Risk	Guardrail or human check
Wrong or invented rule or citation	Answers must cite a retrieved chunk; output_guard blocks uncited rule claims and the bot says "I don't know"; a 20-question test set with expected sections.
Privacy leak	student_id comes from the session, not the message; the tool returns one student's data only; requests about other students are refused.
Prompt injection or jailbreak	input_guard check; retrieved text is passed as data, not as instructions; tool arguments validated by Pydantic.
Unauthorised action	submit_waiver_request is reachable only through human_approval; the advisor can approve, edit or reject; an idempotency key prevents double filing.
Over-promising an outcome	output_guard rejects wording that implies approval; the reply says "sent for review".
Runaway cost or loops	4 agent steps and 25 model calls per session at most.

7. Tech stack, models and cost

Layer	Choice	Why
Chat model and provider	Gemini 2.5 Flash (Google AI Studio, langchain-google-genai)	Free tier, reliable tool calling and structured output, low latency.
Fallback model	Llama 3.3 70B on Groq (langchain-groq)	A different provider, so one outage or rate limit does not stop the app.
Embedding model	BAAI/bge-small-en-v1.5, run locally with FastEmbed	No API cost or rate limit; small and good enough for English regulations.
Vector store	Qdrant (local Docker in development)	Payload filters for document, section and semester; covered in the course.
Other libraries	LangChain, LangGraph with langgraph-checkpoint-sqlite, LangSmith, Pydantic, Streamlit, SQLite, pypdf	One stack the course covered; Streamlit gives student and advisor pages quickly.
Model calls a day (development) and expected cost	About 150–250 calls a day in development (roughly 3 per chat turn). Expected cost: $0 on free tiers; hard cap of $5 if I switch to a paid key.	Embeddings run locally, so only the guard, classify, rewrite and answer calls count.

8. Risks and fallbacks

Risk	Likelihood	What I'll do if it happens
I am not allowed to use the real documents	Medium	Use only the parts the office permits; if refused, write a realistic stand-in regulation set and say so in the demo.
PDF tables and prerequisite charts parse badly	High	Keep prerequisites as structured data in SQLite for the tool; use RAG for prose only; check chunks by hand.
Free-tier rate limits or a model change on demo day	Medium	Fallback model; cached answers for the scripted demo; a recorded backup demo.
interrupt() resume fails or state does not match	Medium	Build a toy version in week 1 and pin library versions; if it fails, use a single-process interrupt and say so honestly.
Scope creep	Medium	Only 2 tools and 2 electives. Finish C1–C7 first, then E4, then E2; no new features.

References

LangGraph documentation: persistence, interrupts and human-in-the-loop. langchain-ai.github.io/langgraph

LangSmith documentation: tracing and evaluation. docs.smith.langchain.com

Qdrant documentation: payload filtering. qdrant.tech/documentation

LangChain documentation: conversational RAG and question rewriting. python.langchain.com